TST Jurisprudence Analysis Pipeline

Capstone Technical Report - CSDS-352

Gustavo Henrique de Jesus da Silva

2026

1 TST Jurisprudence Analysis Pipeline
    1.1 Abstract
    1.2 1. Domain and problem statement
    1.3 2. Dataset
    1.4 3. Architecture
        1.4.1 3.1 What each method does, in plain language
    1.5 4. Interactive interface
    1.6 5. Results
    1.7 6. Limitations
    1.8 7. Future work
    1.9 8. Reproducibility

1 TST Jurisprudence Analysis Pipeline

1.1 Abstract

This report describes an end-to-end data science pipeline that turns the Tribunal Superior do Trabalho (TST) *Livro de Jurisprudência* into a searchable, clustered and quality-controlled corpus of 1058 legal records (*verbetes*: Súmulas, Orientações Jurisprudenciais and Precedentes Normativos). The pipeline parses the official open-access PDF, produces multilingual sentence embeddings, stores them in a

persistent vector database with searchable metadata, reduces and clusters the embedding space, flags low-quality and anomalous documents, and exposes the result through a command line interface and an interactive Streamlit dashboard.

1.2 1. Domain and problem statement

The TST publishes a consolidated book of jurisprudence that is authoritative for Brazilian labour law. The book is distributed as a large PDF, which makes it hard to search semantically, to detect cancelled or duplicated entries, and to understand how its thousands of records group thematically. Legal practitioners and students need a fast, meaning-based way to query this material and an automated way to assess its internal quality.

The capstone addresses three questions:

1. Can a heterogeneous set of legal records be embedded into a shared vector space and clustered into meaningful themes?
2. Can heuristic rules and a supervised classifier reliably separate complete verbetes from cancelled, short or duplicated ones?
3. Can anomalies in the corpus be surfaced automatically for human review?

1.3 2. Dataset

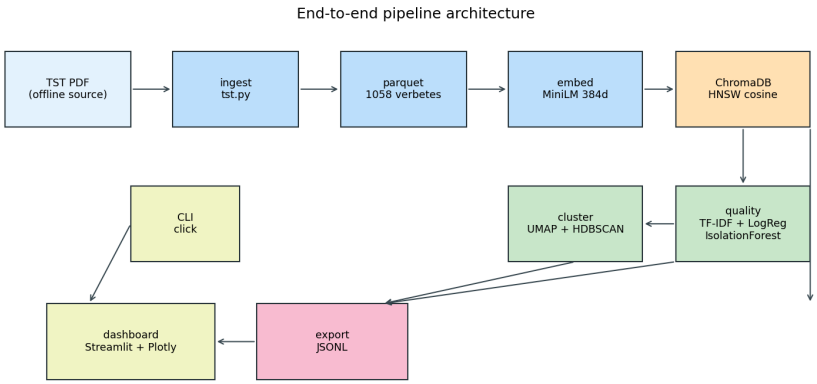
The single data source is the official TST *Livro de Súmulas, Orientações Jurisprudenciais e Precedentes Normativos* (12th edition), downloaded from the TST website (https://www.tst.jus.br/livro-de-sumulas-ojs-e-pns). The PDF is not redistributed in this repository; the reader can fetch it from the official source, where it is published as a public official act (not subject to copyright under Brazilian Law 9.610/98, art. 8, IV).

Parsing the book yields 1058 verbetes with the following distribution by type:

Tipo	Documents
oj (Orientação Jurisprudencial)	703
sumula (Súmula)	235
precedente (Precedente Normativo)	120
Total	**1058**

Text length ranges from 22 to 4204 characters, with a median of 315 characters (figure `figures/fig_text_length.png`). Each record carries searchable metadata: `doc_id`, `tipo`, `orgao`, `codigo`, `tema`, `text`, `source` and `quality_label`.

1.4 3. Architecture



End-to-end pipeline architecture

The system is organised in six stages:

1. **Ingestion** (`src/ingest/tst.py`). PDF parsing with `pdfplumber`, heading detection for each verbete type, removal of historical blocks and the index, and heuristic quality labelling.
2. **Embedding** (`src/embed/model.py`). The `paraphrase-multilingual-MiniLM-L12-v2` sentence transformer produces a normalised 384-dimensional vector for each verbete (`tema` concatenated with `text`).
3. **Vector store** (`src/store/client.py`). Embeddings and metadata are stored in a persistent ChromaDB collection using an HNSW index with cosine distance.
4. **Clustering** (`src/cluster/pipeline.py`). UMAP reduces the vectors to two dimensions and HDBSCAN groups them, assigning each verbete a `cluster_id` and the `umap_x`/`umap_y` coordinates.
5. **Quality control** (`src/quality/`). A TF-IDF + Logistic Regression classifier learns the heuristic labels and re-predicts the quality label for every verbete; an IsolationForest produces a normalised `anomaly_score`.
6. **Search and export** (`src/search/`, `src/export/`). Semantic search with metadata filters over ChromaDB, plus a JSONL export of the enriched dataset.

The CLI (`src/cli/main.py`, built with `click`) exposes every stage individually and as a single `pipeline` command. A `Makefile` provides short aliases.

1.4.1 3.1 What each method does, in plain language

- **Embedding** - a numeric vector that captures the *meaning* of a text; similar meanings produce nearby vectors. We use 384 numbers per verbete, produced by a pre-trained multilingual transformer.
- **Cosine similarity** - a measure of the angle between two vectors: `1.0` means the same direction (very similar), `0.0` means unrelated. Search ranks records by this value.
- **Vector database and HNSW** - a store specialised in nearest-neighbour search over vectors. HNSW is a graph index that finds approximate nearest neighbours quickly, trading a little accuracy for a large speed-up.
- **UMAP** - a dimensionality-reduction method; it projects the 384-dimensional vectors to two dimensions for visualisation while keeping neighbours close, so the structure becomes visible.
- **HDBSCAN** - a density-based clustering algorithm; it groups densely packed points without a preset number of clusters and labels sparse points as *noise*.
- **TF-IDF** - a way to turn text into numbers where rare, informative words weigh more than common ones; “1-2 grams” means single words and adjacent word pairs.
- **Logistic regression** - a simple, fast linear classifier that outputs a probability per class; here it learns which words indicate each quality label.
- **IsolationForest** - an unsupervised anomaly detector; it builds random trees that split the data, and points isolated in few splits receive a higher score.
- **Transfer learning** - reusing a model already trained on a large general corpus instead of training from scratch; this is why 1058 documents are enough for a usable embedding space.

1.5 4. Interactive interface

The exploration interface is a Streamlit dashboard (`src/app/dashboard.py`) backed by Plotly, started with `make dashboard` or `streamlit run src/app/dashboard.py`. It loads the enriched JSONL export and offers three tabs:

- **Search**: semantic search with filters for `tipo`, `orgao` and quality label, ranked by cosine similarity in ChromaDB (with a keyword fallback when the vector store is absent).

- **Cluster map**: an interactive UMAP scatter coloured by HDBSCAN cluster, with per-point hover metadata, a cluster-size chart and per-cluster top terms.
- **Quality and anomalies**: the quality label distribution, the anomaly score histogram and a table of flagged documents filtered by an adjustable threshold.

1.6 5. Results

- **Corpus**: 1058 verbetes (703 OJ, 235 Súmulas, 120 Precedentes Normativos).
- **Clustering**: 2 dense clusters (1009 and 35 verbetes) plus 14 noise points, showing that most of the book shares a common legal vocabulary while a smaller group (historical rural-work precedentes) separates cleanly.
- **Quality classification**: TF-IDF + Logistic Regression reaches 0.906 accuracy and 0.708 macro F1 on a held-out split (212 documents). The model predicts 694 complete, 334 cancelled, 27 short and 3 duplicate verbetes. The detailed evaluation is in the accompanying quality control report.
- **Anomaly detection**: 47 documents (4.4%) score above 0.7; the strongest signals are short historical precedentes that use vocabulary very different from the main corpus.
- **Semantic search**: queries such as `intervalo intrajornada` return the expected verbetes with high cosine similarity.

1.7 6. Limitations

- The heuristic labels are derived from lexical rules, so the classifier inherits their blind spots; the smallest class (`curto`, 5 test samples) is not learned at all (F1 = 0).
- Clustering runs on a 2D UMAP projection for visualisation, which can merge nearby themes; HDBSCAN with the default parameters resolves only two clusters.
- The pipeline ingests the official PDF as well as plain-text files, but only a single collection (the TST book) is validated end to end.
- Results depend on the download of the source PDF; the repository ships the fixtures and the full reproduction commands but not the book itself.

1.8 7. Future work

- Support additional document formats (plain text) and more sources.
- Compare additional embedding models and evaluate retrieval quality with a labelled relevance set.
- Add an active-learning loop so that human corrections feed the classifier.
- Package the dashboard as a container for one-command reproduction.

1.9 8. Reproducibility

```
cd capstone
pip install -e ".[dev]"
make pipeline            # ingest -> embed -> cluster -> quality -
        > export
make stats               # collection statistics
make dashboard           # interactive Streamlit interface
```

The raw PDF is expected at `data/raw/livrointernet12pdf.pdf`. The ABNT sample fixture `tests/fixtures/tst-sample.pdf` allows the pipeline to run offline.